

Human-Centered Permission Boundaries for Personal AI Agents

1. Introduction

Personal AI agents create a privacy problem that ordinary application permissions do not handle well. A user may ask an agent to summarize one reimbursement email, inspect one attached receipt, check one policy page, draft a reply, and maybe send that reply. The useful authority is narrow, but the agent may be connected to a whole mailbox, a file system, a browser, and communication tools. If the interface asks for broad approval such as "allow email and file access," a non-expert user cannot easily tell whether the agent will read only the reimbursement email, open an unrelated tax return, follow malicious webpage instructions, or send private information without a meaningful confirmation step.

The Personal Agent Permission Firewall (PAPF) reframes this problem as task-scoped consent with external enforcement. PAPF does not rely on the LLM to decide what it is allowed to access. A model may help propose task intent or explain a permission prompt, but final authority is compiled, checked, redacted, confirmed, and audited by deterministic components outside the model. This matters for normal users because the system should reduce the amount of hidden authority they must mentally track. Instead of asking the user to trust a broad connector, PAPF turns the current task into a small set of concrete permissions: which data can be read, which tool can be used, what will be redacted, when confirmation is required, and what will be recorded for later review.

This paper draft focuses on the PAPF system design and gives a theoretical argument for why it can be better for non-expert users than broad app-level authorization or prompt-only guardrails. "Better" is used in a bounded sense: under correct task labels, complete mediation, and honest prompt presentation, PAPF reduces hidden over-access opportunities and makes high-risk disclosures explicit. This is not a completed behavioral claim. Whether users actually understand PAPF-style prompts, prefer them, or experience less privacy harm requires the planned non-expert user study.

2. Problem Setting

PAPF targets personal agents that operate across email, files, browser content, and communication tools. The agent receives a user task, proposes or receives structured intent, compiles task-specific authority, and attempts tool calls. The protected resources are synthetic personal data objects such as email threads, receipt files, tax documents, policy webpages, account pages, draft messages, and outbound sends.

The main failure modes are over-collection, cross-context leakage, unauthorized disclosure, prompt-injection-mediated exfiltration, and loss of user agency. Over-collection happens when the agent reads unrelated data because a broad connector is available. Cross-context leakage happens when data from one context, such as a tax return or bank alert, is copied into another context, such as a workplace email. Unauthorized disclosure happens when a send, upload, or chat message releases private data without confirmation. Prompt injection is one mechanism that can trigger these failures by causing untrusted content to ask for off-task access or disclosure.

The user model is a non-expert user who can understand ordinary task language but should not be expected to inspect raw tool-call traces, debug model reasoning, or infer data flows from broad permissions. The

system model is stricter: the policy engine, capability compiler, enforcement mediator, redaction validator, and audit logger are trusted control-plane components. The LLM, retrieved content, webpages, and tool outputs are not trusted to enforce permissions.

The key distinction is between broad authorization and task-scoped authority. Broad authorization gives the agent a large action set, for example access to all emails or files. Task-scoped authority gives the agent only the specific resources and actions justified by the current task, for example `read` on `email_thread_alex_reimbursement`, `read` on `file_receipt_042`, `browse` on `webpage_merchant_reimbursement_policy`, `draft` on a local reply, and `send` only after confirmation.

3. Research Questions

The paper is organized around questions that separate technical feasibility from user-facing privacy outcomes.

RQ	Question	Evidence needed
RQ1	Can PAPF compile natural-language personal-agent tasks into task-scoped capabilities that are narrower than broad connector access?	System design, compiler code, policy records, and trace evaluation.
RQ2	Can PAPF enforce the compiled boundary outside the LLM for reads, browsing, redaction, drafts, sends, and blocked off-task actions?	Policy-engine code, enforcement code, audit traces, and synthetic trace metrics.
RQ3	Why should PAPF theoretically help non-expert users compared with broad app-level authorization?	A formal set-inclusion and complete-mediation argument under explicit assumptions.
RQ4	Can users actually understand and act on PAPF-style prompts?	Planned non-expert user study; not answered by current trace data.
RQ5	Does PAPF introduce unacceptable consent burden or fatigue?	Planned timing, workload, fatigue, and qualitative measures; current trace data only counts prompts.

The current artifact supports RQ1 and RQ2 within a synthetic email-files-browser prototype. RQ3 is addressed by theoretical reasoning in this draft. RQ4 and RQ5 remain empirical questions.

4. System Design

PAPF is a control plane around an AI agent. It has six stages: validate task intent, compile capabilities, evaluate policy, mediate tool execution, validate redaction or confirmation states, and write audit events. The LLM can assist with task interpretation, but it cannot mint authority or bypass enforcement.

The optional intent proposer is deliberately non-authoritative. The code treats model output as raw proposal data that must pass deterministic validation before capabilities can be compiled:

```
# src/papf/intent/proposer.py
def compile_capabilities_from_proposal(
    proposal: IntentProposal,
```

```

    policy_pack: PolicyPack,
    known_data_refs: Iterable[str] | None = None,
) -> CapabilityBundle:
    validation = validate_intent_proposal(proposal,
    known_data_refs=known_data_refs)
    intent = validation.require_intent()
    return compile_capabilities_from_policy(intent, policy_pack)

```

The capability compiler is conservative. It only turns explicit allow rules into capabilities, skips disallowed rules, and refuses to mint broad authority when no explicit resource reference exists:

```

# src/papf/capabilities/compiler.py
for rule in policy_pack.rules:
    if rule.action not in intent.requested_actions:
        continue
    if rule.resource_type not in intent.candidate_resource_types:
        continue
    if rule.consent_level == ConsentLevel.DISALLOW or rule.effect ==
DecisionLabel.DENY:
        continue
    if not rule.allow_refs:
        continue
    for data_ref in rule.allow_refs:
        capability = Capability(
            action=rule.action,
            scope=ScopeSpec.data_ref(rule.resource_type, data_ref),
            purpose_binding=rule.purpose_binding,
            consent_level=rule.consent_level,
        )

```

The capability model also rejects broad request scopes at match time. This is the core of task scoping: a broad tool request cannot satisfy a narrow capability merely because it uses the same tool.

```

# src/papf/capabilities/models.py
def capability_matches_request(capability: Capability, action: Action, scope:
ScopeSpec) -> bool:
    if capability.action != action:
        return False
    if capability.scope.resource_type != scope.resource_type:
        return False
    if scope.is_broad:
        return False
    return capability.scope.matches_data_ref(scope.selector_value)

```

The policy engine then applies complete mediation for each attempted request. It first honors explicit denies, then allows active matching capabilities, narrows a broad request only when the input refs identify exactly one allowed object, requires confirmation for dormant high-risk capabilities, and denies unmatched requests:

```
# src/papf/policy/engine.py
explicit_deny = [rule for rule in rules if rule.effect == DecisionLabel.DENY]
if explicit_deny:
    return _decision(..., DecisionLabel.DENY, ...)

active_match = any(
    capability_matches_request(capability, request.action,
    request.requested_scope)
    for capability in capabilities.active_capabilities
)
if active_match:
    return _decision(..., DecisionLabel.ALLOW, ...)

dormant_match = any(
    capability_matches_request(capability, request.action,
    request.requested_scope)
    for capability in capabilities.dormant_capabilities
)
if dormant_match:
    return _decision(..., DecisionLabel.REQUIRE_CONFIRMATION, ...)

return _deny(run_id, task_id, request.call_id, "MISSING_ACTIVE_CAPABILITY", rules)
```

The enforcement mediator turns policy decisions into execution outcomes. Denials and confirmation requirements are blocked before execution. Redaction-required access is also blocked unless a valid redaction artifact is attached.

```
# src/papf/enforcement/runtime.py
if decision.decision == DecisionLabel.REQUIRE_CONFIRMATION:
    return EnforcementResult(request, decision, ExecutionStatus.BLOCKED)
if decision.decision == DecisionLabel.DENY:
    return EnforcementResult(request, decision, ExecutionStatus.BLOCKED)

if decision.decision == DecisionLabel.ALLOW_WITH_REDACTION:
    if redaction_artifact is None:
        return EnforcementResult(..., execution_status=ExecutionStatus.BLOCKED)
    require_valid_redaction_artifact(...)
```

Redaction evidence is structured and payload-minimizing. The artifact records references and removed field labels, not the raw private payload:

```
# src/papf/enforcement/redaction.py
@dataclass(frozen=True)
class RedactionArtifact:
    redaction_artifact_id: str
    run_id: str
    task_id: str
    call_id: str
```

```

source_refs: tuple[str, ...]
removed_field_labels: tuple[str, ...]
output_ref: str
rationale: str

```

Audit events provide the accountability layer. They link the task, decision, tool call, affected data references, redaction artifacts, and human-readable explanation:

```

# src/papf/audit/models.py
@dataclass(frozen=True)
class AuditEvent:
    audit_event_id: str
    run_id: str
    task_id: str
    event_type: str
    actor: str
    summary: str
    outcome: str
    related_call_id: str | None = None
    related_rule_ids: tuple[str, ...] = ()
    related_data_refs: tuple[str, ...] = ()
    related_redaction_artifact_refs: tuple[str, ...] = ()

```

These modules implement the design goal directly: task interpretation may be flexible, but permission enforcement is narrow, deterministic, auditable, and outside the model.

Theoretical claim 1: PAPF weakly reduces hidden over-access opportunities relative to broad authorization. Let B be the set of actions and data references available under broad app authorization for a session. Let S_t be the set of action-resource pairs compiled by PAPF for task t . By construction, S_t contains only explicit allow references from the policy pack and requested intent actions. Therefore S_t is a subset of B whenever the broad baseline grants connector-wide authority. For any attempted request r not in S_t , `capability_matches_request` returns false, and `evaluate_policy` returns deny unless a valid narrowed or dormant confirmation path exists. `enforce_request` maps deny and confirmation-required decisions to **BLOCKED**. Thus an off-task request that broad authorization could execute is not executed by PAPF without a policy-supported scope or explicit confirmation.

Theoretical claim 2: PAPF lowers the burden on non-expert users to detect invisible over-access. In a broad-authorization design, a user must trust that the model will not use unnecessary parts of the granted connector. In PAPF, off-scope access is blocked by the control plane before execution, so the user does not need to inspect every hidden tool call to prevent unrelated reads. The user's attention is reserved for visible consent states: high-risk confirmation, redaction evidence, or audit review. This is a theoretical usability advantage under the assumption that the prompt accurately exposes the scoped authority and the enforcement layer has complete mediation.

Theoretical claim 3: PAPF does not guarantee lower friction. It can increase interruptions because confirmation gates and redaction evidence create additional user-facing states. The correct theoretical comparison is not "PAPF is always easier"; it is "PAPF trades some friction for a smaller hidden authority surface and more explicit consent points." Whether that tradeoff is preferable for normal users requires empirical study.

5. Discussion

PAPF is better for normal users only under a specific definition of better. It is theoretically better for privacy control because it reduces the invisible action set the user must trust. A broad app-level grant asks the user to approve a large latent capability and then rely on the model to behave. PAPF instead makes the system deny requests that are not justified by the current task. This changes the user's job from supervising every possible hidden action to deciding on a smaller number of structured permission questions.

This theoretical advantage depends on four assumptions. First, task labels and policy rules must correctly identify necessary, unnecessary, and dangerous data. Second, every tool call must pass through the PAPF mediator. Third, the user-facing prompt must faithfully describe the compiled authority and not hide important data flows. Fourth, confirmation and redaction states must be designed so normal users can understand them. The first two assumptions are technical; the last two are HCI questions.

The current trace data supports the technical side but not the user-comprehension side. In the generated 12-trace synthetic run, PAPF records zero false allows and zero over-access, while broad-access and prompt-only baselines each record 16 false allows and an over-access rate of 0.2639. This supports the claim that the prototype can reduce observed over-access in a small synthetic suite. It does not prove that non-expert users understand the prompts or prefer the interaction.

The most important design tradeoff is consent burden. PAPF can prevent invisible over-access by requiring confirmation or redaction evidence before risky actions, but each prompt may interrupt the user. A practical PAPF interface should therefore combine risk-based prompting, progressive disclosure, prompt batching, remembered preferences for low-risk repeated actions, and audit logs for actions that do not need immediate interruption. These strategies are design hypotheses, not proven results.

The paper should therefore make two separate claims. The technical claim is that PAPF provides a concrete architecture for external task-scoped enforcement. The HCI/privacy claim is that this architecture creates a testable interface for user comprehension and consent. The completed artifact supports the first claim. The planned user study is needed for the second.

6. Conclusion

PAPF addresses a central problem for personal AI agents: broad tool access is too hard for normal users to reason about, and prompt-only guardrails leave enforcement inside the component being controlled. PAPF instead compiles the current task into narrow capabilities, mediates every tool call outside the LLM, blocks or narrows off-task requests, requires confirmation for high-risk actions, validates redaction evidence, and records audit events.

The system design gives a theoretical privacy-usability advantage under explicit assumptions. By making the executable authority set smaller than broad authorization and by blocking off-scope requests before execution, PAPF reduces the hidden access surface that a non-expert user must trust. By surfacing redaction and confirmation states, it turns high-risk disclosures into explicit consent moments. This is a proof about enforceable authority and attention allocation, not a proof about actual user comprehension.

The next step is empirical. A user study must test whether PAPF-style prompts help non-expert users recognize necessary access, reject overbroad or unsafe actions, understand redaction and confirmation, and tolerate the added consent burden. Until then, PAPF should be presented as an artifact-backed, theoretically

motivated design for task-scoped personal-agent permissions, not as a deployment-ready or empirically validated permission UX.